

# ABL Customer Statement Intelligence Suite

## Conversational Banking Intelligence Prototype

A production-functional internship prototype combining deterministic statement intelligence, recurring-payment analysis, public-policy retrieval-augmented generation, and grounded language-model assistance inside one unified workspace.

FastAPI

React + Vite

PostgreSQL + pgvector

FastEmbed

Groq

### DEVELOPER

**Daniyal Saqib**

### PRODUCTION-FUNCTIONAL MILESTONE

**10 September 2026**

### PDF REVISION

**23 September 2026**

### PROGRAM

**Allied Bank Internship Program (ABIP)**

### UNIFIED ASSISTANT HARDENING

**22 September 2026**

### HANDOVER WINDOW

**Through 25 September 2026**

**Internship prototype — not an official Allied Bank customer website.**

This independent student project is a synthetic-data demonstration created for educational and internship purposes. Do not upload real customer statements, credentials, or confidential information.

# Current Handover State

The Customer Statement Intelligence Suite is a unified banking-intelligence prototype. The production frontend provides one conversational workspace while the backend keeps three capability-specific intelligence paths: statement intelligence, recurring-payment detection, and public-policy RAG.

The central design rule is that **financial truth remains deterministic**. Python calculates totals, balances, counts, month-specific figures, merchant-specific figures, and recurring-payment candidates. The language model may explain or interpret grounded context, but it is not trusted as the authoritative calculator.

156/156

Automated backend tests passing

v17

Validated Heroku production release

3

Internal intelligence capabilities behind one assistant

## Status Snapshot

| Area                    | Final State                                                                 |
|-------------------------|-----------------------------------------------------------------------------|
| Production code         | a04d7b1 - Harden statement intelligence edge cases                          |
| Current repository main | 47d72d9 - final handover documentation sync                                 |
| Backend production      | Heroku release v17; health endpoint verified ok                             |
| Continuous integration  | GitHub Actions passing                                                      |
| Frontend                | React/Vite production application hosted on Vercel                          |
| Data boundary           | Synthetic/local statement data only; curated public policy information only |

## Live Resources

Frontend

<https://statement-intelligence-suite-abl.vercel.app>

Backend

<https://pure-temple-45004-09958cbb6652.herokuapp.com>

Health

<https://pure-temple-45004-09958cbb6652.herokuapp.com/health>

Swagger

<https://pure-temple-45004-09958cbb6652.herokuapp.com/docs>

Repository

[https://github.com/daniyalsaqib/Statement\\_Intelligence\\_Suite\\_ABL](https://github.com/daniyalsaqib/Statement_Intelligence_Suite_ABL)

**Scope note.** The application is production-functional as an internship prototype. It is not presented as a production banking system and does not connect to live customer accounts or core-banking infrastructure.

# Layered Client-Server Design

The system is implemented as a modular monolith: the user experience is unified, while backend responsibilities remain separated by capability. This avoids unnecessary multi-agent complexity and keeps routing predictable.



## Why This Architecture

### Predictable routing

The LLM does not decide which capability handles a request. Normalized user language is routed deterministically.

### Authoritative arithmetic

Exact statement figures are computed in Python. Model-generated numeric output can be rejected when it conflicts with verified facts.

### Lightweight policy retrieval

MiniLM embeddings run through FastEmbed/ONNX and are stored/searchable inside PostgreSQL through pgvector.

### Unified UX, specialized internals

The frontend feels like one assistant without turning the backend into one unrestricted autonomous agent.

**Architecture in one sentence.** React/Vite on Vercel communicates with FastAPI on Heroku; a deterministic unified router selects statement, recurring-payment, or policy intelligence; Python remains authoritative for financial processing; Groq is used only for grounded language generation; FastEmbed and PostgreSQL/pgvector provide policy RAG.

# Three Intelligence Paths

## 1. Statement Intelligence

Synthetic CSV statements are validated and normalized into transaction records. The backend can deterministically calculate transaction counts, debit/credit counts, total spending, total credits, opening and closing balances, monthly spending/credits, description-specific spending, and largest debit/credit values.

**Multi-turn behavior.** Conversation history improves follow-up UX, but previous assistant messages are never treated as financial truth. A follow-up such as *"And July?"* causes the backend to recalculate the previous reusable statement intent against the newly scoped rows.

## 2. Recurring-Payment Analysis

Repeated outgoing transaction descriptions are detected using deterministic rules and returned as recurring-payment **candidates**. Results include the description, occurrence count, amounts, and dates. The output is intentionally not described as guaranteed subscription classification.

## 3. Public-Policy RAG

Policy questions are embedded using FastEmbed with the 384-dimensional MiniLM model. PostgreSQL + pgvector retrieves relevant public policy context before Groq generates a grounded answer. Unsupported policy questions use fail-closed behavior rather than fabricating unsupported policy facts.

## Unified Assistant Provenance

| Capability             | Method / Provenance             | Authority                                      |
|------------------------|---------------------------------|------------------------------------------------|
| Statement intelligence | verified_statement_intelligence | Verified Python facts                          |
| Recurring payments     | deterministic_recurring_rules   | Deterministic Python rules                     |
| Public policy          | policy_rag                      | Retrieved public context + grounded generation |

## Primary Endpoint

POST /assistant/chat

Specialized statement, recurring-payment, and policy endpoints remain available for backward compatibility; the production frontend uses the unified assistant as its main conversational interface.

# Final Technology Stack

| Layer                    | Technology                                                       |
|--------------------------|------------------------------------------------------------------|
| Frontend                 | React.js, Tailwind CSS, Vite                                     |
| Frontend hosting         | Vercel                                                           |
| Backend                  | FastAPI, Python 3.12                                             |
| Backend hosting          | Heroku                                                           |
| LLM                      | Groq · openai/gpt-oss-20b                                        |
| Embeddings               | FastEmbed / ONNX · sentence-transformers/all-MiniLM-L6-v2 · 384D |
| Database / vector search | PostgreSQL + pgvector · psycopg                                  |
| Version control / CI     | Git, GitHub, GitHub Actions                                      |

## Trust Model

### Trusted / authoritative

- Validated statement rows
- Deterministic Python calculations
- Backend-produced verified facts
- Deterministic recurring-payment rules

### Untrusted / constrained

- User questions
- Transaction descriptions
- Conversation history
- Retrieved policy text as prompt material
- Independent model-generated numbers

## Input Hardening

| Control                   | Bound                   |
|---------------------------|-------------------------|
| CSV upload size           | Maximum 2 MiB           |
| Statement rows            | Maximum 5000            |
| Question length           | Maximum 1000 characters |
| Description length        | Maximum 500 characters  |
| Conversation history      | Maximum 8 messages      |
| Unexpected row fields     | Rejected                |
| Non-finite numeric values | Rejected                |
| Invalid dates             | Rejected                |

**Secret boundary:** Backend API keys, database credentials, and private tokens are not exposed through Vite frontend environment variables.

# Production Verification

## Automated Validation

156

backend regression tests

PASS

GitHub Actions CI

PASS

frontend lint / production build

Regression coverage includes CSV parsing and validation, deterministic financial facts, statement Q&A, month scoping, safe follow-ups, cross-capability conversation history, recurring-payment rules, policy RAG, API failure handling, input hardening, and LLM trust boundaries.

## Current Production Backend

```
Heroku release : v17
Production code: a04d7b1
GET /health    : ok
```

## Final Hardening Examples

| Scenario                   | Verified Behavior                                                                                                         |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Merchant-specific spending | "How much did I spend on Netflix?" returns the exact Netflix debit total rather than the whole statement spending total.  |
| Natural month follow-up    | "And July?" reuses the prior deterministic statement intent and recalculates it for July.                                 |
| Cross-capability history   | A recurring-payment question between statement questions does not replace the earlier reusable statement-spending intent. |
| Reverse chronological CSV  | Opening and closing balances are selected from chronological boundaries rather than raw row order.                        |
| Policy grounding           | Public-policy questions retrieve relevant corpus context and fail closed when support is insufficient.                    |

## Frontend Production Characteristics

The repository records a lightweight and visually stable production frontend, including a median Lighthouse performance score of 93, SEO 100, Best Practices 100, Accessibility 92, approximately 65 KB gzip JavaScript, approximately 6 KB gzip CSS, and cumulative layout shift of 0 during measured validation.

Deployment principle.

Documentation-only changes do not require a Heroku backend redeploy when production code is unchanged. The validated backend release remains v17 on a04d7b1.

# Known MVP Limitations

- Synthetic/demonstration statement data only.
- No live core-banking integration.
- No production customer authentication.
- CSV-oriented statement workflow.
- Recurring-payment results are candidates, not guaranteed subscription classifications.
- The public-policy corpus is intentionally limited and is not automatically synchronized with the Allied Bank website.
- Policy RAG is a demonstration corpus, not a complete representation of every Allied Bank policy.

## Recommended Demonstration Flow

1. Use **Fresh Demo** to generate synthetic multi-month data.
2. Ask: **How much did I spend in August?**
3. Ask: **Which payments keep repeating?**
4. Ask: **And July?**
5. Ask a public-policy question such as the documents required to claim an unclaimed deposit.
6. Change the statement and ask another policy question to demonstrate capability independence.

## Key Handover Explanation

The Customer Statement Intelligence Suite is a conversational banking-intelligence prototype. A React/Vite frontend provides one unified assistant workspace. The FastAPI backend acts as a modular intelligence orchestrator. Financial calculations and recurring-payment detection are deterministic Python operations; open-ended statement interpretation uses verified backend facts with Groq; and public-policy questions use RAG through FastEmbed, PostgreSQL and pgvector. Conversation context improves follow-up UX but is never treated as financial truth.

## Repository Alignment

This PDF revision reflects the current handover state: 156 backend tests, unified assistant architecture, deterministic capability routing, safe multi-turn follow-ups, production code commit a04d7b1, Heroku release v17, and current repository main 47d72d9.

### Internship prototype — not an official Allied Bank customer website.

This independent student project is a synthetic-data demonstration created for educational and internship purposes. Do not upload real customer statements, credentials, or confidential information.